



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 4
Implement Simplex Method for solving linear programming problem
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Aim: To implement Simplex Method for solving linear programming problems.

Theory:

The Simplex Method is an iterative optimization technique used to solve Linear Programming Problems (LPP).

A linear programming problem is of the form:

Maximize or Minimize $Z = c_1x_1 + c_2x_2 + \dots + c_nx_n$

subject to constraints:

$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$

$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2$ and

$x_1, x_2, \dots, x_n \geq 0$

The Simplex Method works by:

1. Converting constraints into a standard form with slack variables.
2. Constructing an initial simplex tableau.
3. Iteratively updating the tableau by selecting an entering and leaving variable.
4. Continuing until an optimal solution is reached.

Procedure/Algorithm:

1. Formulate the Linear Programming Problem (LPP) in standard form.
2. Construct the Simplex tableau by including slack variables.
3. Identify the pivot column (most negative value in the objective row).
4. Identify the pivot row (minimum positive ratio test).
5. Perform row operations to update the tableau.
6. Repeat the process until all coefficients in the objective row are non-negative.
7. Extract the solution from the final tableau.

Code:

```
import numpy as np
def simplex_method(A, b, c):
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    c = np.array(c, dtype=float)
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
num_constraints, num_variables = A.shape
tableau = np.hstack([A, np.eye(num_constraints), b.reshape(-1, 1)])
objective_row = np.hstack([-c, np.zeros(num_constraints + 1)])
tableau = np.vstack([tableau, objective_row])
print("Initial Simplex Tableau:")
print(tableau)
iteration = 0
while np.any(tableau[-1, :-1] < 0): # Continue until no negative values in the objective row
    iteration += 1
    print(f"\nIteration {iteration}:")
    pivot_col = np.argmin(tableau[-1, :-1])
    print(f"Pivot Column: {pivot_col}")
    ratios = tableau[:-1, -1] / tableau[:-1, pivot_col]
    ratios[ratios <= 0] = np.inf # Ignore non-positive values
    pivot_row = np.argmin(ratios)
    print(f"Pivot Row: {pivot_row}")
    pivot_value = tableau[pivot_row, pivot_col]
    tableau[pivot_row] /= pivot_value # Make pivot element 1
    for i in range(len(tableau)):
        if i != pivot_row:
            tableau[i] -= tableau[i, pivot_col] * tableau[pivot_row]
    print("Updated Tableau:")
    print(tableau)
solution = np.zeros(num_variables)
for i in range(num_variables):
    column = tableau[:, i]
    if np.count_nonzero(column[:-1]) == 1 and np.count_nonzero(column[-1]) == 0:
        row_index = np.where(column[:-1] == 1)[0][0]
        solution[i] = tableau[row_index, -1]
print("\nOptimal Solution:")
print(f"x1 = {solution[0]}, x2 = {solution[1]}, Maximum Value = {tableau[-1, -1]}")
c = [5, 4] # Objective function coefficients for Z = 5x1 + 4x2
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

$A = \begin{bmatrix} 6 & 4 \\ 1 & 2 \end{bmatrix}$ # Constraint coefficients

$b = \begin{bmatrix} 24 \\ 6 \end{bmatrix}$ # Right-hand side values

`simplex_method(A, b, c)`

Output:

Initial Simplex Tableau:

$\begin{bmatrix} 6 & 4 & 1 & 0 & 24 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 & 0 & 1 & 6 \end{bmatrix}$

$\begin{bmatrix} -5 & -4 & 0 & 0 & 0 \end{bmatrix}$

Iteration 1:

Pivot Column: 0

Pivot Row: 0

Updated Tableau:

$\begin{bmatrix} 1 & 0.66666667 & 0.16666667 & 0 & 4 \end{bmatrix}$

$\begin{bmatrix} 0 & 1.33333333 & -0.16666667 & 1 & 2 \end{bmatrix}$

$\begin{bmatrix} 0 & -0.66666667 & 0.83333333 & 0 & 20 \end{bmatrix}$

Iteration 2:

Pivot Column: 1

Pivot Row: 1

Updated Tableau:

$\begin{bmatrix} 1 & 0 & 0.25 & -0.5 & 3 \end{bmatrix}$

$\begin{bmatrix} 0 & 1 & -0.125 & 0.75 & 1.5 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 0.75 & 0.5 & 21 \end{bmatrix}$

Optimal Solution:

$x_1 = 3.0, x_2 = 1.4999999999999998, \text{Maximum Value} = 21.$



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Conclusion: The Simplex Method efficiently finds the optimal solution for linear programming problems by iteratively improving the objective function

Answer the following questions:

Q1: Why is the Simplex Method used in Linear Programming?

The Simplex Method is used in Linear Programming because it helps find the best possible solution to problems involving constraints and an objective function. It systematically checks feasible solutions and moves towards the optimal one by improving the objective value at each step. This method is especially useful for solving real-world problems in business, economics, and engineering, where resources like time, money, and materials need to be optimized. Unlike trial-and-error approaches, the Simplex Method follows a structured way of solving equations, making it reliable and efficient for large-scale problems.

Q2: What happens if there is no feasible solution in the Simplex Method?

If there is no feasible solution in the Simplex Method, it means the given constraints cannot be satisfied at the same time. This happens when the constraints contradict each other, making it impossible to find a valid solution. In such cases, the Simplex Method will show that the problem has no solution, and the algorithm will stop without finding an optimal value. This usually happens when the feasible region (the area where all constraints are true) does not exist